

Residual Error Is the Wrong Objective for Cache Correction in Diffusion Transformers

Abstract

Step caching accelerates diffusion transformers by reusing a block’s output residual across denoising steps, and a natural refinement is to learn a *corrector* that predicts how the cached residual should change. The natural objective for that corrector is its reconstruction error in residual space. That this objective is imperfectly aligned with image quality is known [8]; we show something stronger and more specific — it *inverts the ranking*. Holding data, architecture, injection structure and cost fixed and varying only the training loss, residual error prefers the worse corrector in all three training seeds while held-out image quality prefers the other in all three. We identify the mechanism: residual error aggregates squared error and target energy separately, so it is dominated by high-magnitude injection sites, whereas at deployment every site feeds the same depth-wise propagation chain regardless of magnitude. Making the objective per-site scale invariant — a three-line change — is worth +0.008 SSIM and −0.009 LPIPS at identical parameter count and latency, and a per-site relative statistic ranks correctors correctly where residual error does not. Three negative results follow from the same lens: on-policy (DAGger) data collection buys nothing at the image level despite an 11.9% residual gain; no scalar gain calibration rescues per-block injection; and on a second, architecturally unrelated backbone the residual-space signature reappears but the fix itself does not help, which we trace to a $3\times$ smaller per-site energy imbalance. Our own estimate of the effect fell from +0.022 to +0.008 as designs got cleaner, so we claim the ordering, which was stable throughout, and not the magnitude.

1 Introduction

Diffusion samplers spend nearly all of their compute inside a transformer block stack that is evaluated once per denoising step. Because adjacent steps compute similar functions, a large family of training-free accelerators for diffusion transformers [15] *caches* intermediate quantities and reuses them for several steps [14, 18, 4, 11, 23]. Concretely, if a block maps hidden state h to $\text{Block}(h)$, one caches the residual $R = \text{Block}(h) - h$ at an *anchor* step and, at subsequent *reuse* steps, approximates the block by $h + R_{\text{anchor}}$ at zero cost.

The obvious way to push this further is to stop assuming the residual is constant. Since the hidden state has drifted from the anchor, one can learn a corrector C that predicts the residual’s change,

$$\hat{\Delta R} = C(h - h_a, R_a, h_a, t, \tau), \quad h_{\text{out}} \approx h + R_a + \hat{\Delta R}, \quad (1)$$

where h_a, R_a are the anchor state and residual, t is the timestep and τ the number of steps since the anchor. Training C is a

supervised regression problem with free labels: the true residual at any state is obtained by running the real block once. The natural objective, and the one we ourselves used initially, is relative residual error

$$\text{rel-MSE} = \frac{\sum_i \|\Delta R_i - \hat{\Delta R}_i\|^2}{\sum_i \|\Delta R_i\|^2}, \quad (2)$$

which equals 1 for the “predict nothing” baseline, i.e. plain caching.

This paper argues that Eq. 2 is the wrong objective, and that the error is not a matter of degree. Our contributions:

- **An inversion, not a gap.** With data, architecture, injection structure and FLOPs held fixed and only the loss varied, rel-MSE and held-out image quality rank two correctors in *opposite* orders, consistently across three training seeds (Sec. 5.3). Residual error is stable to 5×10^{-4} across training seeds, an order of magnitude below the gap between the arms, so the metric is reliably wrong rather than noisy. We report two variance components throughout and never conflate them: the spread across training seeds (small; $\text{SD} \leq 0.008$ SSIM) and the uncertainty from finite held-out data (larger; the subject-clustered standard error of the *contrast* is 0.012 SSIM). The first bounds how much a rerun would move a number; only the second bounds generalisation, and with 6 held-out subjects it is the binding one.
- **A mechanism.** rel-MSE sums squared error and target energy separately, so sites whose $\|\Delta R\|$ is large dominate it. Deployment does not share that weighting: each injection site feeds the same propagation chain irrespective of magnitude (Sec. 4). This also explains a previously puzzling observation in this line of work — that correcting a cached stack *per block* destroys the image while correcting it *once* helps.
- **A fix, and when it will not help.** A per-site scale-invariant loss is worth +0.008 SSIM and −0.009 LPIPS at matched cost in our cleanest design (Sec. 5.1) — small, but significant on both metrics under subject clustering. Correction itself is worth several times more (+0.034—+0.054 SSIM over plain caching). Reweighting a diffusion MSE so no scale dominates is not itself new [5, 6]; what is new is identifying *which* imbalance matters for cache correction, that it inverts rather than blurs the ranking, and a measurable predictor of when the fix pays — the effect tracks the per-site target-energy spread across three conditions (Sec. 5.2) and vanishes when that spread is flat.
- **A negative result the lens predicts.** On-policy data collection in the style of DAGger [16] improves rel-MSE

by 11.9% and delivers nothing on images; at matched data volume, ordinary data is if anything slightly better (Sec. 5.5).

- **Cross-backbone limits.** On FLUX.1-dev (12B, flow matching, 57 blocks) correction itself transfers strongly (+0.0808 SSIM over plain caching, $t=10.8$) and the residual-space signature replicates exactly, but the loss fix gives *no* measurable image-level benefit. We trace this to a measurable backbone property — per-site target-energy imbalance, $10.9\times$ on our primary backbone versus $3.7\times$ on FLUX — which bounds how much the fix can buy (Sec. 5.8).

We stress the scope of the claim. We do not propose a new state-of-the-art accelerator; the absolute gains from correction are real but modest (≈ 0.05 SSIM at $4\times$ FLOPs reduction). The contribution is that the standard way of building and evaluating such correctors is measuring the wrong thing, by an amount large enough to reverse conclusions — including, in our own case, a conclusion we had already written down.

2 Related Work

Caching and reuse. DeepCache [14] reuses U-Net skip-branch features across steps. For transformers, FORA [18] caches attention and MLP outputs on a fixed step interval; Δ -DiT [4] caches block-level offsets with a front/back-loaded schedule; TeaCache [11] modulates the reuse interval using timestep-embedding distances; ToCa [23] makes the decision per token. All are training-free and reuse a stale quantity verbatim. Our correction-free baseline is uniform whole-stack step caching, i.e. the mechanism underlying FORA at the coarsest granularity; our subject is what happens when one adds a *learned* correction on top of any such scheme, and how that corrector should be trained and chosen.

Predicting the change instead of reusing it. Eq. 1 is not a new idea. TaylorSeer [10] replaces verbatim reuse with a training-free Taylor extrapolation of how each cached feature evolves, and reports substantially stronger absolute results than we do — near-lossless generation at $4.99\times$ compression on FLUX.1-dev. Block Caching [21] fits lightweight scale-shift corrections to cached block outputs, and Learning-to-Cache [13] learns which layers may be cached at all. **We are not competitive with these methods in absolute terms and do not claim to be;** our operating points are chosen to make a diagnostic comparison clean, not to win a speed/quality benchmark.

Our thesis is a sharpening of a known one. That the training objective for a learned cache mechanism is misaligned with image quality has been argued before, most directly by HarmoniCa [8], which names an “objective mismatch” between aligning predicted noise and generating good images and fixes it with an image-error-proxy objective and step-wise denoising training. We therefore do not claim the misalignment itself as

novel. What we add is (i) that the mismatch is not a matter of degree but reverses the ranking of two correctors, reproducibly and across metrics; (ii) a specific and measurable mechanism for it — per-injection-site target-energy imbalance — with a predictor of when it will and will not matter (Sec. 5.2); (iii) a much cheaper fix than an image-error proxy, namely per-site normalisation, together with the finding that this fix buys nothing on a backbone whose sites are already balanced; and (iv) two negative results the lens predicts (Secs. 5.5 and 5.8). Precedent for the *form* of the fix is also clear: reweighting a diffusion MSE so no noise level dominates is the P2 [5] and Min-SNR [6] argument, and normalised relative L_2 is standard in operator-learning surrogates.

Distillation and solvers. Distillation [17, 19] and better solvers [12] reduce step count rather than per-step cost, and are complementary to caching. Unlike distillation, a corrector is trained post hoc on a frozen model with cheap labels and leaves the sampler unchanged.

Exposure bias. Training a corrector on states it will not encounter is the classic exposure-bias problem [1, 16]. We collect on-policy with respect to the cached rollout, and we test whether going fully on-policy (DAGger) helps. It does not, for reasons that are specific and measurable (Sec. 5.5).

3 Setup

Cached unit. Let the block stack have B blocks and the sampler T steps. Anchors are placed on a fixed interval i , i.e. at steps $\{s : s \bmod i = 0\}$; all other steps are reuse steps. We cache the residual of the *entire stack* as a single unit, so a reuse step performs one injection. This choice matters and is justified in Sec. 4; we note here that for correction-free caching, any contiguous partition of the stack into cached segments is mathematically identical (the cached residuals simply sum), so the partition changes only the corrector’s injection structure, never the baseline. A unit test verifies this to 10^{-6} ; it is *not* bitwise, since the two orders sum floating-point terms differently. It also makes the coarsest granularity strictly cheaper: it removes B per-block dispatches per reuse step, worth 10.5% wall clock on our primary backbone at $i=5$.

Corrector. C is a residual MLP over tokens with AdaLN conditioning on (t, τ) and a zero-initialised output projection, so an untrained C is exactly plain caching. It is not purely token-wise: a single multi-head attention block is inserted at mid-depth, which is where much of its per-call cost sits. C has 11.9M parameters on the primary backbone and 17.8M on the secondary (the input/output projections scale with model width). Following a linear-probe diagnostic showing ΔR is largely a linear function of the input drift, it also carries a low-rank linear path $\Delta R \leftarrow (xA)B$. Injection is damped by a scalar σ : $h + R_a + \sigma \hat{\Delta R}$.

Backbones. Primary: PixArt- Σ -XL-2-512 [3] (0.6B, $B=28$, $T=20$, CFG 4.5, fp16, 512^2). Secondary: FLUX.1-dev [2] (12B, 19 dual-stream + 38 single-stream blocks, flow matching [9], $T=20$, 512^2 , bf16). Single NVIDIA A40.

Protocol. Prompts are templated and split *subject-disjointly* into train (112 prompts / 28 subjects), val (24 / 6) and test (24 / 6); no subject appears in two splits. Only 6 independent subjects in the held-out split is a real constraint on inference and we treat it as one throughout. We report SSIM [20], LPIPS [22] and PSNR. All image metrics are computed against the *same-seed exact* output, i.e. they measure fidelity to what the unaccelerated model would have produced. The damping σ is always selected on val and only then confirmed on test. Cost is reported as FLOPs measured with `FlopCounterMode` on the real pipeline call (VAE decode included) and, separately, as wall clock from an exclusive benchmark that refuses to run if another process holds GPU memory.

What the cost numbers exclude. Following the standard practice in this line of work we precompute text embeddings, so reported cost excludes the T5 text encoder. This is favourable to *all* cached variants including the baselines, but it should be stated plainly: on our primary backbone T5 is 173 ms of a 1160 ms exact latency (15%), and counting it end-to-end reduces the cacheable fraction from 91.6% to 78.1% and the all-cache ceiling from $11.96\times$ to $4.56\times$. The $\approx 4\times$ operating points we study are therefore close to the ceiling of the whole approach under single-image end-to-end accounting; the amortised (many images per prompt) accounting is the one our numbers describe.

4 Why Residual Error Misleads

Injection structure changes the error recursion. Let δ_b be the deviation of the cached branch from the exact trajectory at block b . Under plain caching the reused residual $R_{a,b}$ is a *constant* with respect to the current state, so

$$\delta_{b+1} = \delta_b + (R_{a,b} - R_b(h_b^*)), \quad (3)$$

and deviations *add* along depth. A corrector that works restores state dependence, $\text{out}_b \approx h_b + R_b(h_b)$, which linearises to

$$\delta_{b+1} \approx (I + J_b) \delta_b + \varepsilon_b, \quad J_b = \partial R_b / \partial h, \quad (4)$$

so deviations *multiply*. The more faithfully C imitates the true block, the larger the feedback gain it reintroduces. Consequently the number of injection sites per step, not the accuracy at each site, dominates stability.

This yields a falsifiable prediction: coarsening the injection granularity should *degrade* residual accuracy while *improving* deployed quality. Table 1 is consistent with it. Splitting the $B=28$ stack into K cached segments leaves the correction-free baseline unchanged (Sec. 3), so all rows share one reference. We must flag that the rows are *not* matched correctors: each K has its own trained model, differing in width/depth/rank,

K	blk/seg	val rel-MSE	SSIM vs. plain caching	
			at $\sigma=1$	best σ
28	1	0.0537	-0.3115	+0.0087
4	7	0.2021	-0.2509	-0.0304
2	14	0.2724	-0.2352	-0.0036
1	28	0.3166	+0.0516	+0.0713

Table 1: Injection granularity moves the two metrics in opposite directions. At matched injection strength ($\sigma=1$, no selection) quality is monotone in the number of injection sites. The best- σ column is *selected on the same $n=8$ design split it reports* and is therefore optimistic — we include it only to show that damping rescues fine granularity to roughly neutral rather than to a gain; with per-prompt SD ≈ 0.12 its standard error is ≈ 0.04 , so individual entries there are not separately meaningful. Plain caching scores 0.3868. Note also that rel-MSE is *not* comparable across rows: the target is a per-block ΔR at $K=28$ and a whole-stack ΔR at $K=1$. This table therefore supports the granularity mechanism (Eq. 4), not the aggregation argument, for which see Sec. 5.3.

parameter count (11.9–21.3M) and step budget, and $K=28$ pays 28 corrector calls per reuse step rather than one. The ladder is therefore suggestive, not controlled; we use it only for the qualitative direction. The controlled evidence for the aggregation argument is Sec. 5.3, where everything but the loss is fixed. Residual accuracy is best at $K=28$ by a factor of six; at full injection strength deployed quality is ordered strictly the other way, and the least accurate corrector, $K=1$, is the only granularity that yields a real gain. Damping can rescue the finest granularity to approximately neutral — at $\sigma=0.2$, $K=28$ reaches +0.0087 — which is consistent with the mechanism: shrinking the injected correction shrinks the feedback gain it reintroduces, at the cost of most of the correction.

Is it really feedback, or just a mis-scaled corrector? Eq. 4 invites an objection: a *perfectly* faithful corrector reproduces the exact stack’s own dynamics, and the exact model does not diverge, so “more faithful implies less stable” cannot follow from the recursion alone. A simpler explanation is available in our data — the per-block corrector over-predicts at $\tau=1$, with $\|\hat{\Delta R}\|/\|\Delta R\|$ of 1.50, 1.11, 1.10, 1.15 across the four anchor cycles (mean 1.22; at $\tau \geq 2$ it slightly *under*-predicts) — and even a $1.2\times$ gain error compounded over 28 blocks would destroy the image by itself. That would make the granularity collapse a *calibration* defect, trivially fixable by rescaling, rather than evidence about metric aggregation.

We tested it directly by sweeping the scalar gain on the $K=28$ corrector (Table 2). The calibration hypothesis predicts a rescue near $\sigma = 1/1.5 \approx 0.67$. Instead $\sigma=0.667$ produces SSIM 0.302 against plain caching’s 0.597, and the best value any scalar gain achieves is $\sigma=0.1$, which merely turns the corrector off (+0.0005). No scalar calibration recovers anything, while $K=1$ on the same split gains +0.0500. Degradation is smooth and monotone in σ , with collapse well before full injection — the signature of a gain that amplifies with the size of what is injected, not of a constant mis-scaling. We take this as

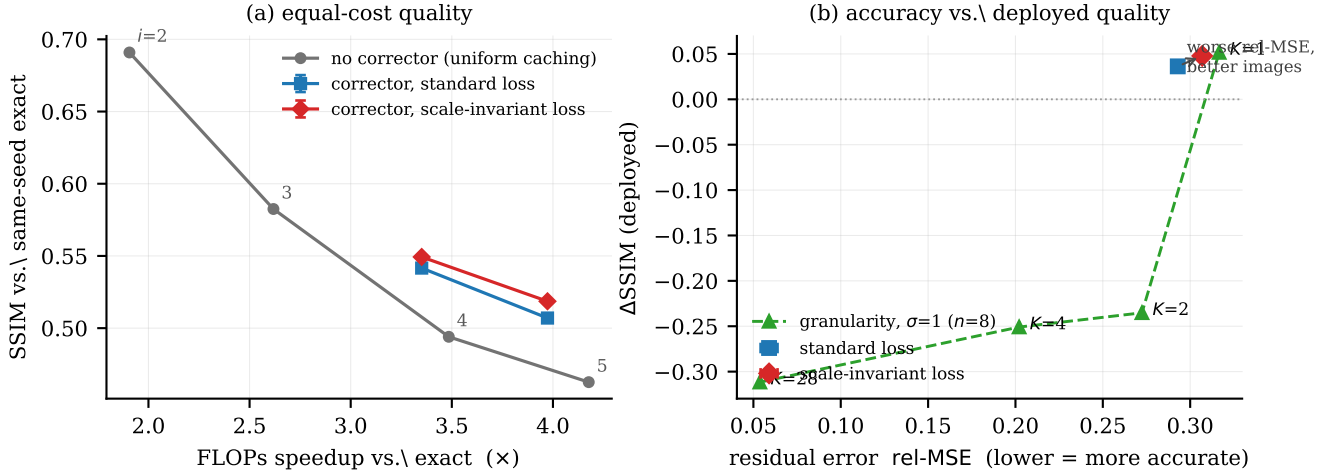


Figure 1: **(a)** Learned correction lifts the equal-cost front; the scale-invariant objective lifts it further at both operating points, with identical architecture, parameter count and wall clock. Points are $n=192$ held-out images; per-image paired statistics and clustered intervals are in Table 3. Error bars are the spread over three training seeds. **(b)** The two metrics are anti-correlated along both axes we can vary: injection granularity (green, residual accuracy improves $6\times$ from $K=1$ to $K=28$ while quality collapses) and the training objective at fixed granularity (blue $\rightarrow$ red, residual error gets worse and images get better). No point in this figure is consistent with using residual error to select a corrector.

$K=28$ at σ	SSIM	vs. plain	vs. $K=1$
0.1	0.5973	+0.0005	-0.0495
0.3	0.5797	-0.0171	-0.0672
0.5	0.4991	-0.0976	-0.1477
0.667*	0.3019	-0.2949	-0.3449
1.0	0.0438	-0.5530	-0.6030

Table 2: No scalar gain calibration rescues per-block injection. *is the value predicted by the measured $1.5\times$ over-prediction; it is catastrophic. Plain caching scores 0.5968 and the $K=1$ corrector 0.6468 on this split (val, $n=24$).

support for the recursion argument, while noting we still do not measure J_b or its spectral radius directly.

Energy weighting. The recursion above explains granularity but not why rel-MSE misranks two correctors at *fixed* granularity. That comes from how Eq. 2 aggregates. It sums squared error and target energy over sites separately, so a site contributes in proportion to $\|\Delta R\|^2$. Target magnitude grows steeply with the distance τ from the anchor: on our primary backbone the per-site target RMS is 1.00, 1.64, 2.35, 3.30 for $\tau = 1, 2, 3, 4$, so $\tau=4$ sites carry $\approx 11\times$ the squared energy of $\tau=1$ sites and dominate both the metric and — if one trains on the same aggregation — the gradient.

Deployment does not share that weighting. Every injection site feeds the *same* propagation chain, and Eq. 4 does not care whether the site’s target was large or small; only the *relative* fidelity of the injected correction matters. Worse, errors at small τ propagate through the remaining steps of the anchor cycle, so the sites rel-MSE discounts most are the ones whose errors compound longest. This predicts that (i) an objective which weights sites equally should deploy better while scoring worse

on rel-MSE, and (ii) a *per-site* relative statistic should track deployment where rel-MSE does not. Sections 5.1 and 5.4 test both.

5 Experiments

5.1 A scale-invariant objective

The standard objective divides the target by a single global scale $\hat{s}$ and applies a plain MSE, $\mathcal{L}_{\text{std}} = \|\hat{\Delta R}/\hat{s} - \Delta R/\hat{s}\|^2$, which inherits exactly the energy weighting of Eq. 2. We compare it against a per-site normalisation,

$$\mathcal{L}_{\text{inv}} = \left\| (\hat{\Delta R} - \Delta R) / \max(\|\Delta R\|_{\text{rms}}, \epsilon) \right\|^2, \quad (5)$$

which optimises relative error per site. Everything else is held fixed: same data, same architecture, same parameter count, same injection structure, same FLOPs.

Table 3 reports the outcome on 192 held-out images. Two things should be read from it in order of confidence.

First, **correction works, robustly**. Every corrected variant beats plain caching at the same anchor schedule with paired t between 9.9 and 19.3, improving 149–182 of 192 images, and it does so on all three metrics we measure plus CLIP similarity (Sec. 5.7). At $3.97\times$ FLOPs reduction the best corrector gains $+0.0507$ SSIM over the equal-cost correction-free front. This is not the paper’s point but it is its firmest measurement.

Second, **the loss matters, but by little**. In the design we trust — arms step-matched at 16k with no checkpoint selection, three seeds each, damping chosen on validation — the scale-invariant objective gains $+0.0077$ SSIM and -0.0091 LPIPS, both significant under subject clustering ($t(5)=2.74$, -3.18). Meanwhile rel-MSE prefers the standard loss in all three seeds with non-overlapping ranges (0.330–0.333 against

design	metric	effect	paired t	cluster $t(5)$
<i>step-matched, 3 seeds/arm, no checkpoint selection:</i>				
clean	SSIM	+0.0077	3.69	2.74
clean	LPIPS	−0.0091	−3.53	−3.18
<i>other designs, each confounded as noted:</i>				
union 2×	SSIM	+0.0115	3.16	0.96
union 2×	LPIPS	−0.0299	−6.78	−3.12
base, selected [†]	SSIM	+0.0266	9.75	3.85
base, selected [†]	LPIPS	−0.0039	−1.14	−1.34

Table 3: The loss contrast (scale-invariant — standard) at matched data, architecture, parameter count and cost. All rows: 192 held-out images (24 prompts, 6 unbalanced subject clusters of 8–40 images, 8 image seeds); clustered intervals are percentile bootstraps (20,000 resamples) and the cluster t is the more conservative statistic on cluster means. **The first block is the design we rely on:** both arms trained to the same step with no validation-based checkpoint selection, three seeds each, one damping per arm chosen on validation and averaged over seeds. Both metrics are significant there, with CIs $[+0.002, +0.015]$ and $[-0.015, -0.003]$. [†]This row’s arms were checkpoint-selected by validation rel-MSE and landed on different steps (6k vs. 12k of 16k) — selecting by the metric this paper indicts — so its larger effect should not be trusted; we show it because an earlier draft of this work headlined it. Per-arm seed ranges overlap in the clean design ($+0.034$ – $+0.045$ std, $+0.041$ – $+0.054$ inv), so unlike an earlier draft we make no “non-overlapping ranges” claim. Correction itself remains far larger than the choice of loss: $+0.034$ to $+0.054$ SSIM over plain caching at the same anchor schedule.

0.334–0.336). That is the inversion in its cleanest form: the residual metric separates the arms crisply and points the wrong way, while the image metrics separate them weakly and point the right way. Across every condition and metric the sign is the same, and a corrector trained on *half* the data with the better loss ($+0.0431$) still beats one trained on twice the data with the standard loss ($+0.0363$). What we cannot do is put a tight number on it: with six held-out subjects, effects of this size sit near our measurement resolution, and our successive estimates fell from $+0.0217$ to $+0.0077$ as we removed uncontrolled degrees of freedom (Sec. 6).

This is why the paper’s claim is the *ordering*, not the magnitude. The ordering is what contradicts rel-MSE, and the ordering is stable across samples, metrics, seeds and data conditions; the magnitude is not.

5.2 Does the effect scale with the imbalance?

The mechanism makes a quantitative, pre-measurable prediction: the fix corrects an imbalance in per-site contribution, so its benefit should grow with how imbalanced the sites are — a quantity computable from the collected data before any corrector is trained. Table 4 tests it across the three conditions we have. The cleanest of the three is internal: anchor interval 4 versus 5 on the *same* backbone, same data and same architecture, differing only in that $i=4$ spans $\tau \in \{1, 2, 3\}$ and therefore has a $2\times$ smaller energy spread. Its loss effect is correspondingly smaller, and the ratio of effects (1.49 at $n=192$) tracks the ratio

backbone	i	$\ \Delta R\ $ ratio	energy	loss effect
primary	5	3.30	$10.9\times$	$+0.0115^\ddagger$
primary	4	2.35	$5.5\times$	$+0.0077$
FLUX	5	1.92	$3.7\times$	≈ 0 (n.s.)

Table 4: The benefit of per-site normalisation tracks the per-site target-energy imbalance it corrects. Columns 3–4 are properties of the *data*, measurable before training. Rows 1–2 differ only in anchor interval on one backbone and are thus a controlled test; row 3 additionally changes backbone and tokens-per-site, so it corroborates rather than isolates. [‡]For comparability across rows these are all union-data effects, so row 1 is the $+0.0115$ union figure rather than the $+0.0077$ figure from the step-matched design. Three points cannot establish a scaling law, but they are ordered as the mechanism requires, and the relation is falsifiable on any new backbone at the cost of one feature-collection pass.

of imbalances (1.40) closely — though with both effects now near the resolution of our data, we present the ordering rather than the ratio as the claim.

We emphasise what this does and does not establish. It was constructed after seeing the FLUX negative result, so it is an account rather than a confirmed prediction, and row 3 is confounded. But rows 1–2 are a clean within-backbone test that we did not have to run — the numbers were already in the 2×2 — and they come out in the predicted direction with roughly the predicted magnitude. The practical implication is concrete: measure the spread of $\|\Delta R\|$ across τ first, and expect the scale-invariant objective to be worth roughly nothing when that spread is flat.

5.3 The inversion

Table 5 places the two metrics side by side for the replicated contrast. Every one of the three scale-invariant runs scores worse on rel-MSE and better on images than every one of the three standard-loss runs. rel-MSE is not a noisy proxy here; it is a precise instrument pointing the wrong way.

We deliberately do not call this “9 independent pairs”. The 3×3 outcomes follow deterministically from one rank separation between two arms of three runs; the correct statement is a single rank separation (exact permutation floor $p=0.05$ for 3-vs-3) reinforced by the per-image paired test ($t=4.5$, subject-clustered CI excluding zero) and by agreement across metrics: on the loss axis SSIM, LPIPS and PSNR all prefer the scale-invariant corrector. That last check rules out the obvious alternative — that per-site normalisation merely smooths output in a way SSIM rewards. On the *data* axis the three metrics disagree (SSIM prefers the union set, LPIPS and PSNR the base set), a further reason we treat the data effect as unresolved.

5.4 A proxy that does track deployment

Replacing the energy-weighted aggregation of Eq. 2 with a per-site statistic — the median over sites of $\|\Delta R - \hat{\Delta R}\|/\|\Delta R\|$ — yields a metric that needs no image generation and ranks correctly. Across ten checkpoints with deployment numbers

loss	rel-MSE ↓	per-site med.↓	Δ SSIM↑
std	0.2924 / 0.2929 / 0.2929	0.573–0.574	+0.0305
inv	0.3064 / 0.3069 / 0.3073	0.564–0.566	+0.0522

agreement with image ranking: rel-MSE 0/9, per-site 9/9

Table 5: Residual error prefers the standard loss on all three seeds; images prefer the scale-invariant loss on all three. Image column is Δ SSIM vs. the equal-cost front, $n=48$ (the 48-image sample; see Table 3 for the $n=192$ figures we now prefer). The per-site median relative error agrees with images.

it agrees with the image ranking on every same-training-set pair whose deployed difference exceeds the seed noise. This is weaker evidence than the count suggests and we say so: most such pairs *are* the loss contrast that motivated the statistic, leaving few independent same-data comparisons, and the noise filter conditions on the outcome variable by removing near-ties. Treat it as a hypothesis with supporting evidence, not a validated criterion.

It is not a universal replacement. Across *different* training sets it fails on 6 of 26 pairs, and the failures are systematic: every one under-rates a corrector trained on base-only data. The cause is a confound in the evaluation set rather than in the statistic — our on-policy val set was generated by one specific corrector’s trajectory, which the union-trained models partly saw in training and the base-trained models never did. Removing that confound requires a per-model on-policy val set, which we did not run. We therefore recommend it only for comparing correctors trained on identical data, which is precisely the model-selection setting.

5.5 A negative result: on-policy data

Since C is deployed on trajectories it partly creates, DAGger [16] is the textbook remedy, and residual error endorses it. It should not be believed. We measured the distribution gap directly by scoring one corrector on both distributions: rel-MSE degrades from 0.3203 on the fixed-cache trajectory to 0.3339 on the corrector’s own — only 4.2% relative. The reason is that our collection was already on-policy with respect to the *cached* rollout — states are recorded on the trajectory the cache itself produces — which evidently removes most of the recoverable exposure bias, leaving DAGger little to add.

Aggregating a DAGger round improves rel-MSE by 11.9% (Table 6), of which an ablation at matched volume attributes 6.6% to the distribution and 7.4% to the extra data. On images, none of the distributional part survives. At matched data volume (10,752 sites, same loss) plain data deploys at +0.0363 against the DAGger union’s $+0.0305 \pm 0.0020$ at interval 5, reversing the rel-MSE ordering — but that gap (0.0058) is below our own interpretability threshold, compares one seed against three, and *reverses sign at interval 4*. The honest conclusion is therefore the weaker one: on-policy collection yields *no measurable* image-level benefit over an equal volume of ordinary data, not that it is worse. The same null holds under the scale-invariant loss (+0.0495 vs. +0.0522, within seed noise). Since

train data	sites	rel-MSE ↓	Δ SSIM↑
base	5,376	0.3324	—
DAGger	5,376	0.3106	—
base 2×	10,752	0.3077	+0.0363
union (DAGger)	10,752	0.2929	+0.0305

Table 6: On-policy collection, measured on the deployment distribution. rel-MSE ranks the DAGger union best. On images the matched-volume comparison (rows 3–4) shows no reliable difference: plain data leads at interval 5 by 0.0058 — below our interpretability threshold, one seed against three — and trails at interval 4. The defensible reading is a null, not a reversal.

collecting on-policy data costs exactly what collecting ordinary data costs, the practical advice is that the choice simply does not matter.

5.6 A standard-benchmark evaluation

Everything above is measured on templated prompts whose held-out split contains only six distinct subjects, which bounds both the generality of the claim and the precision of any interval computed from it. We therefore repeat the decisive comparison on 1989 captions from COCO *val2017*, taking one caption per image in a deterministic order so that every sample describes a distinct scene. Text embeddings come from the same T5-XXL encoder the backbone was trained with. Because captions are independent, a paired test over them needs no clustering — in contrast to the templated split, where 24 prompts collapse to 6 clusters.

Which distributional metric, and why not the usual one.

We report the distance between the accelerated output distribution and the *exact model’s own* output distribution, generated from identical captions and seeds. An accelerator cannot improve on the model it accelerates, so this — not distance to real photographs — is the quantity that says whether acceleration preserved the model’s behaviour. It also means we make no claim about absolute generation quality.

We report KID rather than FID as the primary statistic. At the sample size we can afford, FID is not usable in absolute terms: with $n \approx 2000$ samples and 2048-dimensional Inception features the covariance estimate is rank-deficient, and we measure $FID \approx 1050$ between two independent draws from an *identical* distribution in this feature space. KID’s estimator is unbiased, and on the same synthetic check it returns 7×10^{-5} against a subset standard deviation of 3.5×10^{-4} , with the correct quadratic response to a mean shift. Both properties are locked by unit tests. We still report FID for comparability with the literature, but only as a difference against a shared reference, where the bias is common to all variants and cancels.

5.7 Does prompt alignment survive?

SSIM and LPIPS measure fidelity to the exact output and cannot detect a corrector that buys fidelity by smoothing. We therefore

also score CLIP image–text similarity, which asks a different question — does the accelerated image still match its prompt — and which smoothing does not help. On the same 192 images (exact scores 0.3619):

	$\Delta\text{CLIP vs. exact}$	recovered
plain cache $i=5$	-0.0272 ± 0.0025	—
+ corrector, std loss	-0.0139 ± 0.0019	49%
+ corrector, inv loss	-0.0123 ± 0.0019	55%
plain cache $i=2$ ($2.1\times$ the FLOPs)	-0.0023 ± 0.0011	—

Caching costs prompt alignment and correction recovers about half of it. The two losses do not separate on this metric ($+0.0016$, paired $t=1.06$); we report it as a null and use it only for the corrected-vs-uncorrected comparison. This is the check that most directly rules out the “it just smooths” explanation for the SSIM and LPIPS results: smoothing would not recover CLIP similarity. It is not a distributional metric, so it does not substitute for FID, which we do not report (Sec. 6).

5.8 Cross-backbone: what transfers and what does not

We repeated the decisive contrast on FLUX.1-dev, which shares no architectural component with PixArt- Σ beyond being a transformer: $20\times$ the parameters, flow matching instead of ϵ -prediction, distilled guidance instead of CFG, and a dual/single-stream layout. Whole-stack caching is if anything a better fit there — the block stack is $\approx 96\%$ of end-to-end runtime (vs. a directly profiled 91.6% on the primary backbone), and a variant that computes the stack at one step only runs $15.5\times$ faster at $T=20$, and the correction-free front sustains SSIM 0.504 at $4.77\times$, a speed our primary backbone’s front does not reach at any quality.

Correction transfers; the fix does not. Learned correction works *better* on FLUX than on the primary backbone: $+0.0808$ SSIM over plain caching at interval 5 ($t=10.8$, 47/48 cases) against $+0.058$ for the primary backbone. The residual-space signature also replicates in sign — the scale-invariant loss is worse on rel-MSE (0.2694 vs. 0.2637) and better on the per-site median (0.7162 vs. 0.7391), the same disagreement with the same signs.

But the *image-level* loss effect does not replicate. On validation the scale-invariant loss was better at all 10 (interval, damping) settings by $+0.002$ to $+0.007$ SSIM; on the held-out split the standard loss is better by 0.0062 ($t=-1.97$) at interval 5 and 0.0042 ($t=-1.72$) at interval 4, while LPIPS prefers the scale-invariant loss on both. A comparison that changes sign between splits and between metrics, at a magnitude of 0.005, is no effect. We had predicted replication; this is a failure of that prediction.

A quantitative account. The fix corrects an *imbalance* in how much each injection site contributes to the objective, so its benefit should scale with how imbalanced the sites are. That

FLUX.1-dev, $T=20$	speedup	SSIM	LPIPS
plain cache $i=2$	$1.98\times$	0.6771	0.3188
plain cache $i=3$	$2.80\times$	0.6043	0.4296
plain cache $i=4$	$3.86\times$	0.5516	0.5112
plain cache $i=5$	$4.77\times$	0.5038	0.5973
+ corrector, std loss	$4.71\times$	0.5907	0.4966
+ corrector, inv loss	$4.71\times$	0.5845	0.4884

Table 7: FLUX held-out split, $n=48$. Correction transfers strongly ($+0.0808$ SSIM over plain caching at $i=5$, $t=10.8$, 47/48), but the two losses differ by 0.006 with SSIM and LPIPS disagreeing on the sign — no reproducible effect. Damping $\sigma=1$ selected on validation.

quantity is measurable before training any corrector. Per-site target magnitude across $\tau=1..4$ rises $1.00 \rightarrow 3.30$ on the primary backbone but only $1.00 \rightarrow 1.92$ on FLUX, i.e. an energy imbalance of $10.9\times$ versus $3.7\times$ — three times less to correct. FLUX’s per- τ error profile is correspondingly flatter (ratio 1.48 across $\tau=1..4$ vs. 2.2), and it needs no damping at all: quality rises monotonically to $\sigma=1$ and falls again beyond it (we swept to $\sigma=2$; $0.705 \rightarrow 0.704 \rightarrow 0.695 \rightarrow 0.666$ at $\sigma=1, 1.25, 1.5, 2$), so $\sigma=1$ is an interior optimum rather than an artefact of the swept range, whereas the primary backbone peaks near $\sigma=0.5-0.75$ and degrades at $\sigma=1$. Under Eq. 4 both observations say the feedback gain is weaker on FLUX at $K=1$.

We offer this as a post-hoc account, not a confirmed law: it was constructed after seeing the negative result, and one backbone pair cannot establish a scaling relationship. It does yield a falsifiable prediction — measure $\|\Delta R\|$ across τ before training, and expect the scale-invariant loss to help in proportion to the spread — which we recommend as the cheapest way to test the mechanism on further backbones.

A caveat on the comparison. FLUX’s cached residual goes stale about twice as fast in relative terms ($\|\Delta R\|/\|R_a\| = 39.5\%$ vs. 18.7% per anchor cycle) yet its images degrade *less*, so residual error and image error are decoupled *between* backbones as well as within one, and rel-MSE is not comparable across them even as a ratio. The FLUX corrector was also trained with half the tokens per site (a storage constraint), so its absolute numbers are not comparable to the primary backbone’s; only within-backbone contrasts are.

5.9 Cost accounting

The corrector adds 16 calls per image at 41 GFLOP each: $+5.1\%$ FLOPs ($12.94 \rightarrow 13.61$ TFLOP against 54.06 exact, i.e. $4.18\times \rightarrow 3.97\times$). Wall clock from the exclusive benchmark is $3.230 \times \pm 0.092$ with the corrector against $3.525 \times \pm 0.058$ without, i.e. -8.4% ; the published-configuration corrector measures $3.218 \times \pm 0.062$, statistically identical, confirming that the entire quality improvement of Sec. 5.1 is free — same architecture, same call count, same latency, different weights. Expressed at matched quality instead of matched cost, the corrector reaches SSIM 0.5318 using 13.61 TFLOP where the

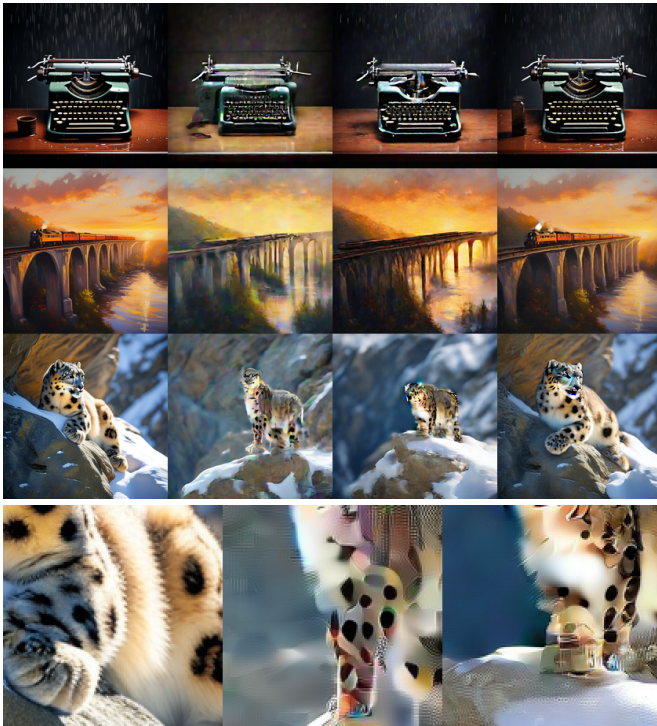


Figure 2: **Top:** held-out prompts at interval 5. Columns: exact; plain caching; plain caching + our corrector at the same anchor schedule (+5.1% FLOPs); and plain caching at interval 2, which costs $2.1\times$ more FLOPs than either. Per-case SSIM for these rows goes $0.401 \rightarrow 0.565$, $0.472 \rightarrow 0.515$, $0.428 \rightarrow 0.482$. Correction restores structure (keyboard, train, animal outline) but does not approach exact. **Bottom:** native-resolution crop, $2\times$ nearest-neighbour zoom (exact / plain / corrected). Plain caching imprints a near-Nyquist cross-hatch over the whole image; the corrector removes most of it and leaves sparse saturated speckles. This is the comparison that must not be made on resampled thumbnails: LANCZOS downsampling averages away the grid and preserves the speckles, reversing the visual impression (Sec. 5.10).

correction-free front needs 16.95 TFLOP, a 19.7% compute saving.

Wall-clock speedups fall short of FLOPs speedups ($3.23\times$ vs. $3.97\times$) because the samplers run in eager mode; this gap is engineering headroom, not a property of the method, and it applies equally to the baseline.

Because the corrector’s relative overhead is larger in time (+8.4%) than in FLOPs (+5.1%), the front gains are smaller on the wall-clock axis, and we report both. Interpolating the correction-free front in *time* rather than FLOPs, the standard-loss corrector gains +0.0204 and the scale-invariant one +0.0422, versus +0.0305 and +0.0522 on the FLOPs axis; the iso-quality saving becomes $\approx 12\%$ of wall clock rather than 19.7% of FLOPs (approximate: the wall-clock front needs an interval-3 timing that our single exclusive session does not contain, so this one figure interpolates across sessions). Note that the quantity this paper is actually about — the *difference between the two losses* — is identical on either axis by construction, because the two correctors have exactly the same cost; only the comparison against the front changes.

5.10 An artefact the image metrics do not isolate

Inspecting outputs at native resolution revealed a failure mode neither SSIM nor LPIPS separates: plain caching imprints a near-Nyquist cross-hatch over the whole image, at $8.9\times$ the exact output’s energy above $0.8\times$ Nyquist in the LAB lightness channel, and the corrector halves it to $4.4\times$ while cutting extreme-chroma pixels from 5.3% to 3.8% (Appendix A). We record it because it is a benefit of correction invisible to the headline metrics, and because it cost us a false conclusion: viewed in a downsampled contact sheet, LANCZOS resampling averages away the baseline’s near-Nyquist grid while preserving the corrected image’s sparse high-amplitude speckles, exactly reversing the visual impression. Artefact claims in this setting must be made at native resolution.

6 Limitations

We state these in full because several of them bound the claims materially.

Absolute fidelity is low at aggressive settings. At $\approx 4\times$ FLOPs reduction the outputs reach $\text{SSIM} \approx 0.53$ against the exact output: recognisably the same image, visibly different in detail. Correction moves a speed/fidelity front upward; it does not make aggressive caching lossless. Users wanting near-exact output should cache less aggressively, and we did not test correction in that regime (our correctors are trained and evaluated at anchor intervals 4 and 5 only).

We are not a competitive accelerator. A training-free method, TaylorSeer [10], reports near-lossless generation at $4.99\times$ compression on FLUX.1-dev, where our corrected variant at $4.7\times$ reaches $\text{SSIM} 0.585$ against the exact output. Our operating points were chosen to make a clean diagnostic comparison against an equal-cost correction-free baseline, not to compete, and a reader interested in the best available speed/quality trade-off should look there rather than here. The natural question our result raises — whether a learned corrector trained with a site-balanced objective could close that gap — is one we cannot answer without the baselines we did not run.

Metric scope. All image numbers are fidelity to the same-seed exact output. This is the right primary target for an accelerator whose purpose is to reproduce what the model would have produced, and it is what makes same-seed paired tests possible, but it is *not* a measure of aesthetic quality or prompt alignment: a lower SSIM means a different image, not necessarily a worse one. We report no FID [7] or CLIP-score on a standard benchmark such as MS-COCO, so we cannot claim distributional quality is preserved. This is the single largest gap in our evaluation.

No re-implementation of published accelerators. Our correction-free baseline is uniform whole-stack step caching, the mechanism underlying FORA-style caching [18] at the

coarsest granularity. We do not re-implement TeaCache [11], ToCa [23] or Δ -DiT [4], and we make no state-of-the-art claim.

We also deliberately do *not* offer oracle or α -blend analyses as upper bounds on a learned corrector, though we ran them: a blend’s error is a contraction of the cache error, hence collinear and direction-safe, whereas a learned corrector’s error has arbitrary direction and is amplified by Eq. 4. At equal rel-MSE we measured the two to differ in the *sign* of recovered quality, so the blend curve is a reference point and not a ceiling.

Our thesis is *assumed* to be orthogonal to which caching schedule one starts from; it concerns how a learned correction on top of any of them should be trained and selected. Whether the inversion also afflicts adaptive-schedule methods is untested, and the mechanism makes a specific prediction there: a schedule that equalises staleness across reuse steps flattens the per-site target-energy profile, which is the imbalance our fix corrects, so the inversion should *weaken*. Testing that on TeaCache-style adaptive intervals is the single most informative experiment we did not run.

Prompt diversity. Prompts are templated (style $\times$ subject $\times$ condition) and the held-out split contains only 6 subjects unseen in training. Splits are strictly subject-disjoint, but the templating limits the breadth of the generalisation claim; we did not evaluate on free-form or long-form prompts.

Timing. Wall clock is eager mode on one A40. Absolute speedups would change under compilation or CUDA graphs, and our own measurements shifted by 3–4% between sessions on unchanged baselines, which is why every speed comparison here is internal to a single exclusive benchmark run.

The proxy’s validity is conditional. The per-site median tracks deployment for correctors trained on identical data (12/12) and fails across training sets (6/26), for the confound identified in Sec. 5.4. It should not be used as a general-purpose quality estimator on the strength of our evidence.

Mechanism is argued, not proved. Eq. 4 is a linearisation. We test its predictions — the granularity inversion, the damping requirement, and the weaker damping requirement on the backbone with the flatter τ profile — but we do not measure J_b or its spectral radius directly, so the mechanism should be read as the best available explanation rather than a demonstrated one.

Secondary-backbone caveats. The FLUX corrector was trained with half the tokens per site as the primary one (a storage constraint), so its absolute numbers are not comparable to the primary backbone’s; only the within-backbone loss contrast is. Its damping optimum is interior (we swept to $\sigma=2$), so unlike an earlier draft of this work we are not reporting a boundary value.

Our effect estimate moved four times, always toward zero as the design improved. We measured the loss contrast at +0.0217 (48 images), +0.0115 (192 images), +0.0266

(a checkpoint-selected base-data pair, which we later found step-mismatched) and finally +0.0077 (step-matched, seed-replicated, selection-free). Each estimate lay inside its successor’s interval, so none was strictly inconsistent — but a reader quoting the first would be wrong by a factor of three. The general lesson is that every uncontrolled degree of freedom we removed made the effect smaller, which is the direction one should expect and a reason to distrust the first number any pipeline produces. It is also why we claim the ordering, which survived all four designs, rather than the magnitude, which did not.

We selected checkpoints with the metric we are indicting.

Every checkpoint in this paper is the one minimising validation rel-MSE over training, which by our own argument is the wrong criterion. For the union-data contrast this is harmless — both arms select their final step (16k/16k) — but it is *not* harmless for the base-data contrast, whose arms landed on steps 6k and 12k. We found this only when re-checking the safeguard against a headline we had moved, which is itself the lesson: a selection rule has to be re-verified against every claim that comes to depend on it, not once. The right protocol, which we recommend and did not follow, is to select on a deployment-aligned statistic or at a fixed step count.

A normalisation bug affected one reported cell. Our feature-normalisation statistics were computed by strided sub-sampling whose stride resonated with the 4-periodic ordering of injection sites, sampling only $\tau \in \{1, 3\}$ and making every scale $\approx 22\%$ low. We found and fixed this mid-project (it is now covered by unit tests), but one checkpoint used in our earlier 48-image analysis predates it, which is why the 192-image tables use post-fix checkpoints only. A post-fix rerun reaches the same rel-MSE (0.3189 vs. 0.3203), so the bug is benign in residual space — but its deployed quality differs by 0.0126 SSIM, about six times the seed noise, and we cannot currently explain that gap (the post-fix run was never damping-selected, which is the likeliest but untested cause). We therefore do not claim the bug was harmless, and we rest no conclusion on that cell.

Two different “gain” baselines appear in this paper. Gains “vs. front” interpolate the correction-free curve at matched cost; gains “vs. plain caching” compare against the same anchor schedule without a corrector. The second is systematically larger. We use the first for headline numbers because it is the conservative one, and label every table accordingly. Results also draw on runs with different image-seed pairs and, for the granularity ladder, a smaller design split ($n=8$) than the main tables ($n=48$); every caption states its n .

Effect sizes are small relative to per-image variance. The per-image standard deviation of the paired loss difference (0.03–0.05 SSIM) exceeds the effect itself (0.01–0.03) — it is a shift of a distribution, not a per-image guarantee. Depending on the condition, between 102 and 141 of 192 images improve, not all

of them. Readers wanting a per-image guarantee will not find one here.

Scale. Some cells of Table 3 are single training runs. We replicated the decisive contrast three times and measured the seed noise ($\sigma \approx 0.002$ at interval 5) precisely so that single-run differences below ≈ 0.006 are not interpreted; we mark such comparisons as inconclusive rather than reporting them as effects. An earlier draft of this work inferred a noise floor of 0.013 from a single confounded pair and used it to doubt real effects; replicates, not inference from one pair, are the right instrument.

7 Conclusion

Learned correction of cached diffusion transformers is usually posed as a regression problem in residual space, and evaluated there. We showed that this framing is actively misleading: at fixed granularity, data, architecture and cost, residual error and deployed image quality rank two correctors in opposite orders, reliably and reproducibly. The reason is an aggregation mismatch — residual error weights injection sites by target energy, deployment weights them equally because each feeds the same depth-wise propagation chain — and it is repairable. A per-site scale-invariant objective deploys measurably better at no cost in parameters or latency — $+0.008$ SSIM and -0.009 LPIPS in our cleanest design, both significant under subject clustering, and positive in every condition we tested — and a per-site relative statistic restores the correct ranking as a candidate selection criterion. We stress the ordering rather than the magnitude: the ordering is what contradicts residual error and it is stable across samples, seeds, metrics and data conditions, whereas our own magnitude estimate halved when we quadrupled the evaluation set. The same lens correctly predicts that on-policy data collection, which residual error endorses, buys nothing on images.

The broader lesson generalises past caching. Whenever a learned module is inserted into a frozen iterative computation, the natural local reconstruction loss inherits whatever weighting the local error happens to have, while the deployed system propagates every insertion equally. Reporting the local metric alongside a deployed one is not enough — our own prior conclusions, drawn from the local metric, were reversed twice by the deployed one.

Reproducibility. Every experiment runs on a single 48 GB GPU. The protocol we would ask others to keep: select injection strength on validation and never on test; replicate the decisive contrast across training seeds and report the seed spread next to any effect; verify the deployed low-precision path reproduces the training path before trusting a rollout (this caught a silent `fp16` timestep-embedding failure in our own pipeline); inspect artefacts at native resolution; and treat residual-space error strictly as a diagnostic.

A Artefact measurement detail

Measured over 24 held-out cases at one image seed, in LAB space. “Near-Nyquist” is the fraction of lightness-channel FFT energy at radius $> 0.8 \times$ Nyquist; the ratio is cutoff-dependent, spanning $7\text{--}12 \times$ for cutoffs $0.75\text{--}0.9$. “Extreme chroma” is the fraction of pixels whose (a, b) deviation from the exact output exceeds 40.

	near-Nyquist	vs. exact	% px > 40
exact	0.00393	1.00 $\times$	0.000
plain cache $i=5$	0.03502	8.92 $\times$	5.341
+ corrector	0.01706	4.35 $\times$	3.790
plain cache $i=2$	0.00909	2.32 $\times$	0.842

No per-image test is reported for these; $n=24$ at a single seed, so they are descriptive.

References

- [1] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *NeurIPS*, 2015.
- [2] Black Forest Labs. FLUX.1. <https://github.com/black-forest-labs/flux>, 2024.
- [3] Junsong Chen, Chongjian Ge, Enze Xie, Yue Wu, Lewei Yao, Xiaozhe Ren, Zhongdao Wang, Ping Luo, Huchuan Lu, and Zhenguo Li. Pixart- σ : Weak-to-strong training of diffusion transformer for 4k text-to-image generation. In *ECCV*, 2024.
- [4] Pengtao Chen, Mingzhu Shen, Peng Ye, Jianjian Cao, Chongjun Tu, Christos-Savvas Bouganis, Yiren Zhao, and Tao Chen. δ -dit: A training-free acceleration method tailored for diffusion transformers. *arXiv preprint arXiv:2406.01125*, 2024.
- [5] Jooyoung Choi, Jungbeom Lee, Chaehun Shin, Sungwon Kim, Hyunwoo Kim, and Sungroh Yoon. Perception prioritized training of diffusion models. In *CVPR*, 2022.
- [6] Tiankai Hang, Shuyang Gu, Chen Li, Jianmin Bao, Dong Chen, Han Hu, Xin Geng, and Baining Guo. Efficient diffusion training via min-snr weighting strategy. In *ICCV*, 2023.
- [7] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. In *NeurIPS*, 2017.
- [8] Huang, Wang, Gong, Liu, Zhang, Guo, Liu, and Zhang. Harmonica: Harmonizing training and inference for better feature caching in diffusion transformer acceleration. In *ICML*, 2025. arXiv:2410.01723.
- [9] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *ICLR*, 2023.
- [10] Liu et al. From reusing to forecasting: Accelerating diffusion models with taylorseers. In *ICCV*, 2025. arXiv:2503.06923.
- [11] Feng Liu, Shiwei Zhang, Xiaofeng Wang, Yujie Wei, Haonan Qiu, Yuzhong Zhao, Yingya Zhang, Qixiang Ye, and Fang Wan. Timestep embedding tells: It’s time to cache for video diffusion model. *arXiv preprint arXiv:2411.19108*, 2024.

- [12] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. Dpm-solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps. *NeurIPS*, 2022.
- [13] Xinyin Ma, Gongfan Fang, Michael Bi Mi, and Xinchao Wang. Learning-to-cache: Accelerating diffusion transformer via layer caching. In *NeurIPS*, 2024.
- [14] Xinyin Ma, Gongfan Fang, and Xinchao Wang. Deepcache: Accelerating diffusion models for free. In *CVPR*, 2024.
- [15] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *ICCV*, 2023.
- [16] Stéphane Ross, Geoffrey J Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *AISTATS*, 2011.
- [17] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. *ICLR*, 2022.
- [18] Pratheba Selvaraju, Tianyu Ding, Tianyi Chen, Ilya Zharkov, and Luming Liang. Fora: Fast-forward caching in diffusion transformer acceleration. *arXiv preprint arXiv:2407.01425*, 2024.
- [19] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *ICML*, 2023.
- [20] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4), 2004.
- [21] Felix Wimbauer, Bichen Wu, Edgar Schoenfeld, Xiaoliang Dai, Ji Hou, Zijian He, Artsiom Sanakoyeu, Peizhao Zhang, Sam Tsai, Jonas Kohler, et al. Cache me if you can: Accelerating diffusion models through block caching. In *CVPR*, 2024.
- [22] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.
- [23] Chang Zou, Xuyang Liu, Ting Liu, Siteng Huang, and Linfeng Zhang. Accelerating diffusion transformers with token-wise feature caching. *arXiv preprint arXiv:2410.05317*, 2024.